Palm OS Programming Recipes

Adding HotSync Logging to a Conduit

The Palm™ Conduit Development Kit includes tools that provide you detailed debugging options during the development cycle. However, after you ship a conduit, you cannot assume that end users have access to these tools. The only assumption you can safely make about end users' desktop environment is that the HotSync® software is present. To help you support end users' problems in the field, your conduit can take advantage of the HotSync Log API to provide verbose logging when HotSync Manager is launched with special command-line arguments.

This recipe shows you how to implement verbose logging in your conduit in the following steps:

1. Extract the `HotSync.exe` command-line arguments.
2. Write entries to the HotSync log.
3. Save the HotSync log.

At the end, this recipe suggests some additional logging functions you can use and describes how to view verbose entries in the HotSync log. **1. Extract the HotSync.exe Command-line Arguments**

To support verbose HotSync logging, you must extract the `HotSync.exe` command-line arguments. The following code extracts the command-line parameters using the `GetCommandLine` Windows API function.

```
#define LOG_NORMAL 0x0000
#define LOG_L1 0x0001
#define LOG_L2 0x0002
#define LOG_VERBOSE 0x0004

DWORD dwLogLevel;

struct logOpt_t
{
   char *szCmd;
   DWORD dwCmdValue;
} logOptions[] = {
   _T("V"), LOG_VERBOSE,
   _T("L1"), LOG_L1,
   _T("L2"), LOG_L2
};

void ProcessCmdLineParameters()
{
   CString csCmdLine(GetCommandLine());
   csCmdLine.MakeUpper();

   CString csDelim = _T("-");

   dwLogLevel = LOG_NORMAL;
   int nSize = sizeof(logOptions)/sizeof(logOpt_t);

   for (int n=0; n<nSize; n++)
   {
```

```
      if (csCmdLine.Find(csDelim + logOptions[n].szCmd) != -1)
         dwLogLevel |= logOptions[n].dwCmdValue;
   }
}
```

2. Write Entries to the HotSync Log

After detecting the `HotSync.exe` command-line arguments, you can now implement code that logs conduit progress only under specified conditions. For example, the following code writes an entry to the HotSync log if a call to `SyncOpenDB` returns an error:

```
// Open the handheld database.
retVal = SyncOpenDB( pName, 0, hDataBase );
if ( retVal )
{
   if ( retVal == SYNCERR_FILE_NOT_FOUND )
   {
      if (dwLogLevel & LOG_VERBOSE)
      {
         // The verbose (-v) flag was used, so write
         // the error to the HotSync log.
         LogAddEntry("SyncOpenDB returned SYNCERR_FILE_NOT_FOUND",
                slText, true);
      }
   }
}
```

Additionally, the HotSync Log API contains the `LogAddFormattedEntry` function, which supports the same formatting instructions as do standard C library functions. For example, the following code writes an entry to the HotSync log if a call to `SyncPurgeAllRecs` returns an error:

```
int retval = -1;
CString dataString = "%s failed in the function %s with error %d";
CString csFunction = "DatabaseClearRecs()";
CString csErrFunction;

// Body of the function...

// Clear all the existing records in the database.
retVal = SyncPurgeAllRecs( hDataBase );
if ( retVal )
{
   // Store the name of the function that failed.
   csErrFunction = "SyncPurgeAllRecs()"

   if (dwLogLevel & LOG_VERBOSE)
   {
      // The verbose (-v) flag was used, so write the error
      // to the HotSync log.
      LogAddFormattedEntry(slRemotePurgeFailed, true,
         dataString, csErrFunction,
         csFunction, retval);
   }
}
```

3. Save the HotSync Log

Because the HotSync log can quickly expand with verbose (-v) logging enabled, your conduit might need to save the contents of the HotSync log to a specified file. The following code saves the contents of the HotSync log to a local file:

```
LPCTSTR pcsLogFile = "HSBackupLog.txt";
retval = LogSaveLog(pcsLogFile);
```

Additional HotSync Log API Functions

The following HotSync Log API functions are also available and documented in the *C/C++ Sync Suite Reference*:

- `LogBuildRemoteLog` retrieves an abbreviated HotSync log from the handheld.
- `LogCloseLog` closes the HotSync log.
- `LogGetWorkFileName` returns the file name that HotSync Manager is using as the working log file.
- `LogInit` initializes the existing HotSync log or creates a new log.
- `LogTestCounters` returns the number of error activity types in the HotSync log.
- `LogUnInit` destroys the HotSync log object.

Viewing Verbose HotSync Logging

The conduits that ship with the Palm Desktop software detect `HotSync.exe` command-line arguments and provide verbose logging. To examine the usage of the HotSync Log API in the these conduits or others, follow these steps:

1. If the HotSync Manager application is running, exit the application.
2. Run the HotSync Manager application with the verbose (-v) option from a command line (alternatively, click the Start button, and then click Run). Then type:

   `<hotsyncpath>\hotsync.exe -v`

   where `<hotsyncpath>` is the location of the HotSync software (for example, `C:\Palm`).

3. Perform a HotSync operation.
4. Examine the HotSync log. (In the Windows toolbar, click the HotSync Manager icon, and then click View Log... .)